CO661 Class 1 - Java concurrency basics

This class covers basics of multi-threaded shared-memory concurrency in Java. It covers some additional practical concerns specific to Java not discussed in the lectures.

Task 1 - Creating threads

Create a simple Java program that prints `Hello` and one second later prints `World!` where this second message is printed by a separate thread. You will need:

- A Java program `Class1.java` with public class `Class1` and some other class, e.g., `Worker`;
- for `Worker` to inherit from `Thread` and implement `public void run();`
- a `Thread.sleep(long millis)` static method in `Worker`;
- to watch out for `InterruptedException`;
- to `start()` the worker thread from the main method.

Task 2 - Implementing `Runnable` rather than extending `Thread`

Java does not allow multiple inheritance (i.e., we cannot has a class that `extends` multiple classes, or that `extends` multiple times), therefore the general advice in Java is to try to use *interfaces* as much as possible. In Java, the `Runnable` interface[1] can be used in conjunction with the `Thread` class to avoid having to extend `Thread`. A class implementing `Runnable` must provide a public `run` method that returns `void` just as we implemented before, e.g.,

```
class Worker implements Runnable {
    public void run() { /* TODO */ }
}
```

A "runnable" thread then needs to be attached to a thread which can then be started via `start` (or waited for via `join`), e.g.

```
Thread t = new Thread(new Worker());
t.start();
```

Rewrite your solution to Task 1 using this approach.

Task 3 - What to do with `InterruptedException`

Threads can be *blocked* which means they are waiting for some external event and are currently not making any progress e.g., due to `Thread.sleep` (waiting for a timer) or waiting for another thread to finish when using `.join()`, or waiting to be unblocked (which we see later). A blocked thread can be interrupted in Java by calling the `.interrupt()` method on the thread object, e.g.

```
Thread t = new Thread(new Worker());
t.start();
t.interrupt();
```

This starts a thread and immediately interrupts it. This mechanism is seen as a "polite" way to externally shut down a thread (rather than just what happens when the JVM is stopped entirely) and can be used to break deadlocks. However, we will usually focus on trying to ensure that deadlocks can never occur anyway.

By catching and handling `InterruptedException`, we can provide some "cleanup" code in the case that this task was interrupted. If we do not have any cleanup code we want to perform, (and

[1] `https://docs.oracle.com/javase/10/docs/api/java/lang/class-use/Runnable.html`

maybe we don't anticipate any interrupts happening our code anyway), the recommended practice is to *reinterrupt* the current thread.

What does it mean to reinterrupt a thread? Inside a thread, there is an `isInterrupted` flag.[2] If the runtime detects that this flag is true then an `InterruptedException` is thrown and the flag is immediately set back to false. Reinterrupting means setting this flag back to true by `Thread.currentThread().interrupt()`. This allows code higher up the stack to be aware of the interruption.

So rather than doing nothing in the `catch` block, the recommendation[3] is to at least do:

```
try {
    // ...
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
}
```

Edit your previous solutions to use this technique.

Task 4 - Client-server with a critical section in the server

Download `Server.java` from the Moodle. In this task you will write three classes; you can put them all in the same file (e.g., `Main.java` with public class `Main` providing the static `main` method).

1. Write a class `MyServer` that implements the `Server` interface, providing a method to generate messages (strings) based on some input string which is inserted into a template string drawn from a list of templates, e.g. the server should include some list:

   ```
   private LinkedList<String> templates;

   public MyServer() {
      this.templates = new LinkedList<String>();
      this.templates.add("Hello <name>.");
      ...
   ```

   (recall you can replace substrings within a string using the `replace` method).

   Each template should only be used once, and so the server can run out messages, represented by `generateMessage` returning an `Optional`[4] value.

2. Implement a client class which has a string associated with it (the client's "name") and access to the server (both passed via its constructor). The client should be runnable as a thread and should repeatedly asks the server to generate messages based on its name, until no more messages are provided by the server.

3. Create a main class to bring all these together, with a server and a number of clients running in parallel on their own threads.

4. Consider what are the possible race conditions in your implementation? Can you insert some judicious "sleep" statements to expose racey behaviour?

5. Enforce mutual exclusion using one of the approaches seen in Lecture 2 (or Lecture 3 if you want and are doing this after watching Lecture 3).

[2] https://docs.oracle.com/javase/10/docs/api/java/lang/Thread.html
[3] https://www.ibm.com/developerworks/java/library/j-jtp05236/index.html?ca=drs-
[4] https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/Optional.html